Topic 2 – ETL and Data Warehousing

A. Data Warehouse Fundamentals

1. What is the difference between OLTP and OLAP?

Interview question: Explain OLTP and OLAP with examples.

Answer:

OLTP stands for Online Transaction Processing. It is used to manage day-to-day business transactions, such as inserting, updating and deleting records.

OLAP stands for Online Analytical Processing. It is used to analyze large volumes of historical data and generate business reports.

OLTP	OLAP
Handles daily transactions	Handles analytical queries
Frequent INSERT, UPDATE, DELETE	Mostly SELECT operations
Current operational data	Historical and consolidated data
Usually highly normalized	Often dimensional or denormalized
Fast individual transactions	Efficient analytical queries
Example: Banking application	Example: Banking analytics warehouse

Practical example:

Imagine a banking application.

- When a customer transfers ₹10,000, the transaction is recorded in an OLTP database.
- When management wants to analyze total transactions across all branches over the last five years, that analysis happens in an OLAP environment.

Remember: OLTP is for running the business; OLAP is for analyzing the business.

2. What is a Data Warehouse?

Answer:

A Data Warehouse is a centralized repository that stores integrated, historical data collected from multiple source systems for reporting and analytical purposes.

For example, a banking organization may collect data from:

- Customer management systems.
- Account databases.
- Transaction systems.
- Loan management systems.

This data is processed through ETL pipelines and stored in a data warehouse such as BigQuery.

The business can then use this consolidated data for reporting, analytics and decision-making.

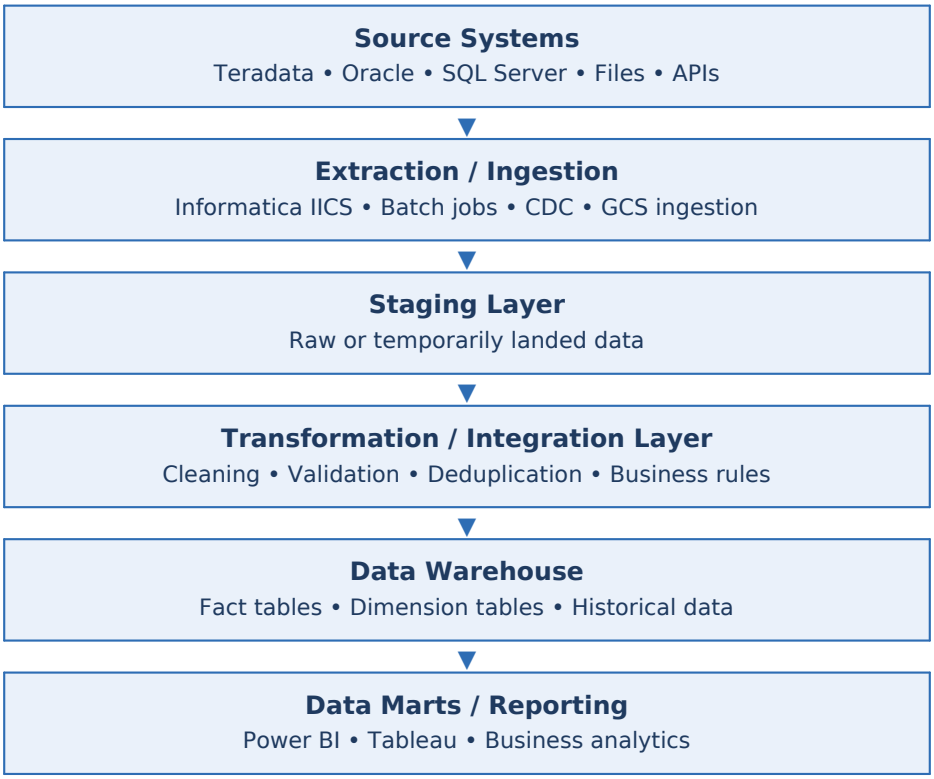
Four traditional characteristics of a data warehouse:

1. Subject-oriented – Organized around subjects such as customers, accounts and transactions.
2. Integrated – Data from different systems is brought into a consistent format.
3. Time-variant – Historical data is maintained for analysis over time.
4. Non-volatile – Primarily used for analytical reading rather than frequent transactional updates.

3. Explain Data Warehouse Architecture.

Answer:

A typical data warehouse architecture consists of multiple layers.



The main objective is to collect data from different systems, transform it into a consistent structure and make it available for analytical consumption.

4. What is a Staging Area?

Answer:

A staging area is an intermediate storage layer where data is temporarily stored after extraction from source systems and before being loaded into the final data warehouse.

It is useful for:

- Initial data landing.
- Data cleansing.
- Data type conversion.
- Duplicate identification.
- Data validation.
- Error investigation.
- Restarting failed loads.

Example:

Suppose we extract 10 million records from Teradata.

Instead of loading them directly into the final BigQuery table, we can first load them into a staging table, validate the records and then load the cleaned data into the final target.

5. What is a Data Mart?

Answer:

A Data Mart is a smaller, subject-oriented subset of a data warehouse designed for a specific business department or function.

For example:

- Finance Data Mart – Revenue, expenses and financial transactions.
- Customer Data Mart – Customer profiles and customer activity.
- Sales Data Mart – Orders, products and sales performance.

A data mart makes it easier for individual departments to access the data relevant to their business requirements.

6. What is a Fact Table?

Answer:

A fact table stores measurable business events or transactions along with references to related dimension tables.

It generally contains:

- Foreign keys referencing dimensions.
- Numeric measures.
- Transaction or event-related information.

Example: Banking transaction fact table

Transaction_ID	Customer_Key	Account_Key	Amount	Transaction_Date_Key
101	501	1001	5000	20261101
102	502	1002	7500	20261101
103	501	1001	2000	20261102

Here, `Amount` is a measure, while the other keys help connect the transaction to descriptive information.

7. What is a Dimension Table?

Answer:

A dimension table stores descriptive information that provides context to the measures in a fact table.

For example, a customer dimension may contain:

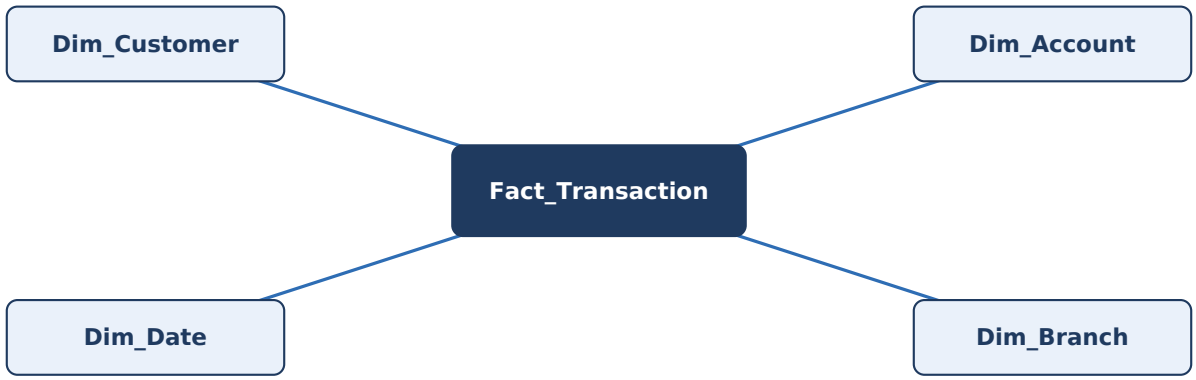
Customer_Key	Customer_ID	Customer_Name	City
501	C101	Rahul	Pune
502	C102	Anil	Bangalore

The fact table tells us what happened and the dimension tables help us understand who, what, where and when.

8. What is a Star Schema?

Answer:

A Star Schema is a dimensional modelling approach where a central fact table is directly connected to multiple dimension tables.



For example:

- Fact_Transaction
- Dim_Customer
- Dim_Account
- Dim_Date
- Dim_Branch

It is called a Star Schema because the structure resembles a star.

Advantages:

- Simple structure.
- Fewer joins.
- Easy reporting and analytics.
- Good query performance for many analytical workloads.

9. What is a Snowflake Schema?

Answer:

A Snowflake Schema is an extension of a Star Schema where dimension tables are further normalized into related tables.

For example, instead of storing the city, state and country directly in one customer dimension, we can separate geographical information into related dimension tables.

Difference:

Star Schema	Snowflake Schema
Less normalized dimensions	More normalized dimensions
Fewer joins	More joins
Simpler structure	More complex structure
Often easier for reporting	Can reduce repeated dimension attributes

10. What are Surrogate Keys and Natural Keys?

Answer:

A natural key is an identifier that comes from the business or source system.

A surrogate key is an artificially generated key used internally in the data warehouse.

Example:

Customer_ID (Natural Key)	Customer_Key (Surrogate Key)
C101	1001
C102	1002
C103	1003

Why do we use surrogate keys?

- Source identifiers may change.
- Multiple source systems may use overlapping identifiers.
- Surrogate keys support historical tracking.
- They help implement Slowly Changing Dimensions, especially Type 2.

11. What is the Grain of a Fact Table?

Answer:

The grain defines exactly what one row in a fact table represents.

For example:

If a transaction fact table stores one row per individual banking transaction, then its grain is:

"One row represents one individual transaction."

If a sales fact table stores daily sales totals per product per store, its grain is:

"One row represents the total sales of one product in one store on one day."

Why is grain important?

It determines the level of detail in the fact table and helps avoid duplicate or incorrectly aggregated data.

B. ETL Concepts

1. What is ETL? Explain the process.

Answer:

ETL stands for Extract, Transform and Load. It is a process used to collect data from different source systems, transform it according to business requirements and load it into a target data warehouse.

The three stages are:

1. Extract: Retrieve data from source systems such as Teradata, Oracle, SQL Server or flat files.
2. Transform: Clean, validate, filter, join and apply business rules to the extracted data.
3. Load: Load the processed data into the target warehouse, such as BigQuery.

Example:

Suppose we need to migrate customer transaction data from Teradata to BigQuery.

- Extract transaction records from Teradata.
- Transform the data by applying business rules, removing duplicates and standardizing formats.
- Load the validated data into BigQuery.

2. What is the difference between ETL and ELT?

Answer:

Both ETL and ELT are data integration approaches, but they differ in where the transformation takes place.

ETL	ELT
Extract, Transform, Load	Extract, Load, Transform
Transformation happens before loading into target	Data is loaded first, then transformed
Uses an ETL engine for transformations	Uses target platform processing capabilities
Common in traditional ETL platforms	Common in cloud data warehouses
Example: IICS transformations before target loading	Load raw data into BigQuery, then transform using SQL

Practical example:

In an ETL approach, IICS can extract data, apply transformations and load the final result into BigQuery.

In an ELT approach, we first load raw data into BigQuery and then use SQL transformations to populate the final warehouse tables.

Interview tip: BigQuery is well suited to ELT because it can perform large-scale transformations using its distributed processing architecture.

3. What is the difference between Full Load and Incremental Load?

Answer:

A full load loads the complete dataset from the source into the target.

An incremental load processes only the new or modified records since the previous successful load.

Example:

Assume the source table contains 10 million records.

- Full load: Process all 10 million records.

- Incremental load: Process only the 50,000 new or modified records identified for the current run.

Full Load	Incremental Load
Processes all records	Processes changed records
Takes more time for large datasets	Usually faster
Useful for initial migration	Useful for regular daily loads
Can replace or refresh target data	Usually inserts or updates target data

Important: Incremental loading must also handle deleted records if the business requirement includes source deletions.

4. What is Batch Processing?

Answer:

Batch processing means collecting and processing data in groups at scheduled intervals rather than processing every record immediately when it arrives.

For example, a banking organization may process all transactions received during the day through a scheduled ETL workflow at midnight.

Advantages:

- Suitable for large volumes of data.
- Supports scheduled processing.
- Efficient resource utilization.
- Easier to coordinate dependent workflows.

Example from ETL projects:

A Control-M job can trigger an IICS workflow every day after the source data files are available.

5. What is CDC (Change Data Capture)?

Answer:

CDC stands for Change Data Capture. It is a technique used to identify changes made to source data and transfer those changes to the target.

It generally captures three types of changes:

1. Insert – New record added.
2. Update – Existing record modified.
3. Delete – Existing record removed.

Example:

Suppose a customer table initially contains:

Customer_ID	Name	City
101	Rahul	Pune

The customer moves to Bangalore.

After the update:

Customer_ID	Name	City
101	Rahul	Bangalore

CDC identifies this change and allows the target system to update the corresponding customer record.

Common CDC approaches:

- Database transaction logs.
- Timestamps or modification columns.
- Triggers.
- Change tables.
- Source-system CDC tools.

Interview follow-up: Is CDC the same as incremental loading?

No. Incremental loading is the broader process of loading only changed data. CDC is one way to identify those changes.

6. What is Watermark-Based Loading?

Answer:

Watermark-based loading is an incremental loading technique that uses a control value to identify which records should be processed in the next run.

The watermark is commonly a timestamp or sequence number.

For example, assume the last successful load processed records up to:

`2026-11-03 23:59:59`

The next run identifies records with an update timestamp later than the stored watermark, subject to the agreed boundary and overlap logic.

Example SQL:

```
SELECT *
FROM SOURCE_TABLE
WHERE UPDATE_TIMESTAMP > TIMESTAMP '2026-11-03 23:59:59';
```

After successful processing, the watermark is updated in the control table.

Important interview points:

- Update the watermark only after successful processing.
- Handle late-arriving records.
- Consider records with identical timestamps.
- Use an overlap window when necessary.
- Make the target load idempotent to avoid duplicates.

Why should we not update the watermark before the job completes?

If the job fails after the watermark is advanced, records that were not successfully processed may be skipped in the next run.

7. What is Source-to-Target Mapping?

Answer:

Source-to-target mapping is a document or specification that defines how each source column maps to the corresponding target column.

It usually includes:

- Source table and column.
- Target table and column.
- Source and target data types.
- Transformation logic.
- Default values.
- Lookup or join conditions.
- Business rules.

Example:

Source Column	Target Column	Transformation
CUST_ID	CUSTOMER_ID	Direct mapping
FIRST_NAME	CUSTOMER_NAME	Concatenate with LAST_NAME
DOB	DATE_OF_BIRTH	Date conversion
STATUS	CUSTOMER_STATUS	Apply business mapping

Why is it important?

It acts as a reference for ETL development, testing, migration and business validation.

8. What is Data Validation?

Answer:

Data validation is the process of checking whether the data meets the expected technical and business requirements.

Common validation checks include:

- Mandatory field validation.
- NULL validation.
- Data type validation.
- Duplicate validation.
- Date and format validation.
- Range validation.
- Business-rule validation.

Example:

If `CUSTOMER_ID` is mandatory, we should verify that the target does not contain unexpected NULL customer IDs.

```
SELECT COUNT(*)
FROM CUSTOMER
WHERE CUSTOMER_ID IS NULL;
```

9. What is Data Reconciliation?

Answer:

Data reconciliation is the process of comparing source and target data to ensure that the migrated or processed data is consistent.

Common reconciliation checks include:

- Source and target record counts.
- Sum of transaction amounts.
- Minimum and maximum dates.
- Missing records.
- Extra records.
- Duplicate business keys.
- Column-level data comparison.

Example:

Validation	Source	Target
Record count	1,000,000	1,000,000
Total transaction amount	50,00,000	50,00,000
Distinct customer count	50,000	50,000

If the values match, that is a positive reconciliation result. However, we should still perform key-level or column-level checks where required.

Difference between validation and reconciliation:

Validation checks whether data is correct according to defined rules. Reconciliation compares two datasets or systems to identify differences.

10. What is Error Handling in ETL?

Answer:

Error handling is the process of identifying, recording and managing failures that occur during ETL execution.

Common ETL errors include:

- Database connection failures.
- Invalid data types.
- Missing source files.
- Constraint violations.
- Duplicate records.
- Transformation errors.
- Target load failures.

A good error-handling approach should:

1. Capture the error message.
2. Identify the failed workflow or transformation.
3. Log the failed record or execution details.
4. Separate rejected records when appropriate.
5. Notify the support team.
6. Support safe recovery or rerun.

Example:

If a source file contains an invalid date value, the ETL process can identify the invalid record, log the reason and handle it according to the agreed reject-record policy.

11. What is Restartability and Recovery in ETL?

Answer:

Restartability means an ETL process can be restarted after failure without unnecessarily repeating completed work or creating inconsistent results.

Recovery means bringing the ETL process and target data back to a consistent state after a failure.

For example, suppose a workflow has three stages:

1. Extract source data.
2. Load staging table.
3. Load final target table.

If the third stage fails, we should identify whether staging completed successfully and restart from a safe point rather than blindly re-extracting and inserting everything again.

How do you achieve restartability?

- Maintain job execution status.
- Use control tables and watermarks.
- Use staging tables.
- Maintain batch IDs.
- Implement idempotent loading.
- Log processed and rejected records.
- Validate data before marking a batch successful.

C. Slowly Changing Dimensions (SCD)

This is one of the most important data warehousing interview topics. Interviewers frequently ask you to explain the differences between SCD Type 1 and Type 2 using examples.

1. What is a Slowly Changing Dimension?

Answer:

Slowly Changing Dimensions are techniques used in data warehousing to manage changes in dimension attributes over time.

For example, a customer may change their address from Pune to Bangalore.

The data warehouse must decide whether to overwrite the old address or preserve the historical address.

SCD techniques help manage these changes.

2. Explain SCD Type 0.

Answer:

SCD Type 0 means the original dimension value is retained and not updated, even if the source value changes.

For example, if a customer's original registration date is recorded as 1 January 2024, that value remains unchanged.

It is useful for attributes that should remain fixed.

3. Explain SCD Type 1.

Answer:

SCD Type 1 overwrites the existing dimension record when an attribute changes.

It does not maintain historical values.

Example:

Before the change:

Customer_ID	Customer_Name	City
C101	Rahul	Pune

The customer moves to Bangalore.

After the Type 1 update:

Customer_ID	Customer_Name	City
C101	Rahul	Bangalore

The old city, Pune, is no longer retained in that dimension record.

When do we use Type 1?

When historical tracking is not required, such as correcting a spelling mistake or updating a non-historical attribute.

4. Explain SCD Type 2.

★ **Very important**

Answer:

SCD Type 2 maintains complete historical records by inserting a new row whenever a tracked dimension attribute changes.

Instead of updating the existing record, we expire the old record and insert a new record with a new surrogate key.

Example:

Initially:

Customer_Key	Customer_ID	City	Start_Date	End_Date	Current_Flag
1001	C101	Pune	2024-01-01	9999-12-31	Y

Customer moves to Bangalore on 1 November 2026.

After the Type 2 change:

Customer_Key	Customer_ID	City	Start_Date	End_Date	Current_Flag
1001	C101	Pune	2024-01-01	2026-10-31	N
1002	C101	Bangalore	2026-11-01	9999-12-31	Y

What happens internally?

1. Identify the customer using the natural/business key.

2 Compare the incoming attribute values with the current dimension record.

3 If there is a tracked change, expire the existing record.

4 Insert a new row with a new surrogate key.

5 Mark the new row as the current record.

Why do we use a new surrogate key?

Because historical fact records may need to refer to the old version of the dimension, while new fact records should refer to the new version.

5. Explain SCD Type 3.

Answer:

SCD Type 3 maintains limited historical information by storing the previous value in an additional column.

For example:

Before the change:

Customer_ID	Current_City	Previous_City
C101	Pune	NULL

After the customer moves to Bangalore:

Customer_ID	Current_City	Previous_City
C101	Bangalore	Pune

Unlike Type 2, Type 3 does not create a new dimension row for every change. It stores limited history in the same record.

6. What are Effective Start Date and Effective End Date?

Answer:

These columns are commonly used in SCD Type 2 to identify the validity period of each dimension record.

- Effective Start Date: Date from which the dimension record becomes valid.
- Effective End Date: Date until which the dimension record remains valid.

For example:

Customer_ID	City	Start_Date	End_Date
C101	Pune	2024-01-01	2026-10-31
C101	Bangalore	2026-11-01	9999-12-31

This allows us to identify which customer address was valid on a particular date.

7. What is a Current Record Indicator?

Answer:

A current record indicator identifies which dimension record represents the latest valid version.

For example:

- `Y` means current record.
- `N` means historical record.

In SCD Type 2, when a new version is inserted, the previous record's current indicator is changed to `N`, and the new record is marked as `Y`.

Example query:

```
SELECT *
FROM DIM_CUSTOMER
WHERE CURRENT_FLAG = 'Y';
```

This retrieves the current customer records.

8. How do you manage historical records in SCD Type 2?

Answer:

We maintain multiple records for the same natural key, each representing a different version of the dimension.

Whenever a tracked attribute changes:

1. Identify the current record.
2. Update its effective end date.
3. Change its current flag to `N`.
4. Insert the new version.
5. Generate a new surrogate key.
6. Set the new record's current flag to `Y`.

This allows historical reporting without losing previous values.

9. What is a Late-Arriving Dimension?

Answer:

A late-arriving dimension occurs when a fact record arrives before its corresponding dimension record is available in the data warehouse.

For example:

A transaction arrives for customer `C101`, but the customer record has not yet been loaded into the customer dimension.

The fact record cannot immediately find its corresponding dimension key.

How do you handle it?

Common approaches include:

1. Hold the fact record until the dimension arrives.
2. Insert an inferred or placeholder dimension record.
3. Update the inferred record when the actual dimension data becomes available.

An inferred member is useful when we need to load the fact without waiting for the complete dimension information.

Interview tip: If you use a placeholder dimension, explain how you later update it with the actual customer attributes and preserve the correct surrogate-key relationship.

D. Real-World ETL Scenarios

These are especially important for senior ETL and data engineering interviews. Focus on explaining your approach step by step.

1. How would you load 10 million records daily?

Answer:

For loading 10 million records daily, I would first understand the source data volume, available extraction method, target architecture and SLA requirements.

My approach would be:

1. Extract the required records using incremental loading or CDC rather than unnecessarily extracting the entire dataset.
2. Land the extracted data in GCS or a staging table.
3. Validate the incoming data.
4. Remove duplicates using business keys.
5. Apply the required transformations.
6. Load the processed records into BigQuery.
7. Use `MERGE` for insert and update operations where appropriate.
8. Perform record-count and aggregate reconciliation.
9. Maintain batch status and execution logs.

For performance, I would consider parallel processing, partitioning, clustering and query optimization.

Follow-up: Would you always use incremental loading?

Not necessarily. For an initial migration, a full load may be required. For daily processing, incremental loading is generally more efficient if the source supports reliable change identification.

2. How do you identify new and updated records?

Answer:

I would use a combination of business keys and change-identification mechanisms.

Common approaches include:

- Last updated timestamp.
- CDC.
- Sequence numbers.
- Change tracking tables.
- Hash comparison.

For example, suppose the source contains:

ID	Name	Update_Timestamp
101	Rahul	2026-11-01
102	Anil	2026-11-04

If the last successful watermark was 3 November, record 102 can be identified as changed based on its timestamp.

For new and updated records, I would compare the incoming business keys with the target:

- Key not found in target: Insert.

- Key found with changed attributes: Update.
- Key found with unchanged attributes: No action.

3. How do you handle duplicate business keys?

Answer:

First, I would identify why duplicates exist.

Possible reasons include:

- Duplicate records in the source.
- Multiple updates for the same key.
- Incorrect join conditions.
- Reprocessing of previously loaded records.
- Missing deduplication logic.

If the requirement is to retain the latest record, I can use `ROW_NUMBER()`.

```
SELECT *
FROM STAGING_CUSTOMER
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY CUSTOMER_ID
    ORDER BY UPDATE_TIMESTAMP DESC
) = 1;
```

This keeps one latest record per customer, assuming the ordering column reliably identifies the latest version.

I would then load the deduplicated records into the target and validate the business-key uniqueness.

Important: If two records have identical timestamps, use an additional deterministic ordering column, such as a source sequence or change version.

4. How do you handle missing source records?

Answer:

First, I would identify whether the missing records are genuinely deleted from the source or simply not extracted.

I would investigate:

1. Source extraction filters.
2. Incremental watermark conditions.
3. Source file completeness.
4. ETL rejected records.
5. Source-to-target record differences.
6. Whether the source supports deletion tracking.

If the source record is deleted, the target handling depends on the business requirement.

Possible approaches include:

- Physical deletion from the target.
- Soft deletion using a status flag.

- Historical tracking through SCD Type 2.

I would never delete target records just because they are absent from one incremental extract. Absence from an incremental batch does not necessarily mean deletion.

5. What happens if an ETL job fails halfway?

Answer:

I would first identify the exact stage where the job failed and whether the target was partially updated.

For example, suppose the workflow has three stages:

1. Extract data from Teradata.
2. Load data into BigQuery staging.
3. Merge data into the final BigQuery table.

If stage 3 fails, I would check whether staging completed successfully and whether the final table was partially modified.

Then I would:

- Review execution logs.
- Check batch status.
- Identify processed and unprocessed records.
- Validate the target state.
- Restart from a safe point.
- Perform reconciliation after recovery.

The goal is to recover without losing data or creating duplicates.

6. How do you prevent duplicate loading during reruns?

Answer:

I would design the ETL process to be idempotent.

Some techniques include:

- Maintaining a unique business key.
- Using batch IDs.
- Tracking successful execution status.
- Deduplicating staging data.
- Using BigQuery `MERGE`.
- Updating watermarks only after successful processing.
- Implementing restartable workflows.

For example, if the same batch is executed twice, a properly designed `MERGE` should update existing records or insert missing records rather than blindly inserting duplicates.

However, I would also ensure the staging input contains one appropriate record per merge key.

Final interview revision: What you must be able to explain

Concept	What you should remember
OLTP vs OLAP	Transactions vs analytics
Data warehouse	Centralized historical analytical repository
Staging	Intermediate landing and processing area
Fact table	Measures and dimension keys
Dimension table	Descriptive business attributes
Star schema	Central fact with dimensions
Snowflake schema	Normalized dimensions
Surrogate key	Warehouse-generated identifier
Grain	What exactly one fact row represents
ETL vs ELT	Transform before vs after loading
CDC	Capture inserts, updates and deletes
Watermark	Track incremental processing progress
Reconciliation	Compare source and target
Restartability	Recover safely after failure
SCD 0	Retain original value
SCD 1	Overwrite existing value
SCD 2	Insert new historical version
SCD 3	Store limited previous values
Late-arriving dimension	Fact arrives before dimension

Test yourself – 10 interview questions

Try answering these without looking at the notes. These are the questions I would prioritize for your interview.

1 What is the difference between a fact table and a dimension table?